Homework 5 - Charlie Caravati

In [19]:
```
import numpy as np
from numpy.linalg import inv
from numpy.linalg import norm

import math
```

1.

$f(x,y) = 3x^2 - y^2 = 0$

$g(x,y) = 3xy^2 - x^3 - 1 = 0$

(a)

$$\begin{bmatrix} x_{n+1} \\ y_{n+1} \end{bmatrix} = \begin{bmatrix} x_n \\ y_n \end{bmatrix} - \begin{bmatrix} \frac{1}{6} & \frac{1}{18} \\ 0 & \frac{1}{6} \end{bmatrix} \begin{bmatrix} f(x_n, y_n) \\ g(x_n, y_n) \end{bmatrix}, n = 0, 1, 2, \ldots$$

In [15]:
```
def equations(x,y):
    f = 3 * x**2 - y**2
    g = 3 * x * y**2 - x**3 - 1
    return np.array([f,g], dtype = float)

M = np.array([[1/6, 1/18],[0, 1/16]], dtype=float)

tol = 10e-5
max_iter = 25
```

In [16]:
```
cur = np.array([1, 1])

print (f"Step 0: x = {cur[0]}, y = {cur[1]}")

for n in range(max_iter):

    F_val = equations(cur[0], cur[1])

    next_val = cur - M @ F_val

    change = np.linalg.norm(next_val - cur)

    cur = next_val

    print (f"Step {n+1}: x = {cur[0]}, y = {cur[1]} with change {change}")

    if change < tol:
        print (f"Converged on step {n+1} to x = {cur[0]}, y = {cur[1]}")
        break
```

```
Step 0: x = 1, y = 1
Step 1: x = 0.6111111111111112, y = 0.9375 with change 0.39387919201353416
Step 2: x = 0.5495835059918458, y = 0.9135559668102708 with change 0.06602244252603125
Step 3: x = 0.5259918908093882, y = 0.9004294470450471 with change 0.026997589300972243
Step 4: x = 0.5153506031163272, y = 0.8920634768564516 with change 0.01353611690858799
Step 5: x = 0.5099957543364605, y = 0.8862233005395764 with change 0.007923513416876974
Step 6: x = 0.507013672179177, y = 0.8819113212197048 with change 0.005242707282291152
Step 7: x = 0.5051833921689826, y = 0.8786186961146779 with change 0.003767134852639477
Step 8: x = 0.50396048019974, y = 0.8760541637757469 with change 0.002841186337067635
Step 9: x = 0.5030880439145654, y = 0.8740334727562944 with change 0.0022009855219390106
Step 10: x = 0.5024367508216707, y = 0.872430435831336 with change 0.001730291904746872
Step 11: x = 0.5019362228801337, y = 0.8711535669653303 with change 0.0013714672147863362
Step 12: x = 0.5015447011544868, y = 0.8701339884516852 with change 0.0010921673897074068
Step 13: x = 0.5012352248654866, y = 0.8693185971096086 with change 0.00087214598215374
Step 14: x = 0.5009891001482106, y = 0.8686658496759069 with change 0.0006976077613231519
Step 15: x = 0.500792658943123, y = 0.8681429552593287 with change 0.0005585765103769612
Step 16: x = 0.5006355435569445, y = 0.867723888760047 with change 0.0004475510869098886
Step 17: x = 0.5005097252060442, y = 0.8673879234826867 with change 0.0003587518989707099
Step 18: x = 0.5004088940896011, y = 0.8671185157155412 with change 0.00028765858068457344
Step 19: x = 0.5003280504585099, y = 0.8669024410471654 with change 0.00023070317510108602
Step 20: x = 0.5002632132795151, y = 0.866729117708291 with change 0.0001850536127142412
Step 21: x = 0.5002112034651375, y = 0.8665900724543995 with change 0.00014845404481302957
Step 22: x = 0.5001694778925498, y = 0.8664785169114138 with change 0.00011910357920146602
Step 23: x = 0.500136000002838, y = 0.8663890104561593 with change 9.556241223289008e-05
Converged on step 23 to x = 0.500136000002838, y = 0.8663890104561593
```

This appears to converge but does so linearly, which is not very quick.

(b)

The Jacobian of the system evaluated at (1,1) is $\begin{bmatrix} 6x & -2y \\ 3y^2 - 3x^2 & 6x \end{bmatrix} = \begin{bmatrix} 6(1) & -2(1) \\ 3(1)^2 - 3(1)^2 & 6(1) \end{bmatrix} = \begin{bmatrix} 6 & -2 \\ 0 & 6 \end{bmatrix}$ gives determinant = 6(6) - (-2)(0) = 36

which gives $J(1,1)^{-1} = \frac{1}{36}\begin{bmatrix} 6 & 2 \\ 0 & 6 \end{bmatrix} = \begin{bmatrix} 1/6 & 1/18 \\ 0 & 1/6 \end{bmatrix}$

The numerical matrix in the iteration uses the inverse of the jacobian at the initial guess to avoid recomputing the matrix inverse at each step.

(c)

In [20]:
```
def evalF(x):

    F = np.zeros(2)
```

```
    F[0] = 3 * x[0]**2 - x[1]**2
    F[1] = 3 * x[0] * x[1]**2 - x[0]**3 - 1

    return F

def evalJ(x):

    J = np.array([[6*x[0], -2*x[1]],
        [3*x[1]**2 - 3*x[0]**2, 6*x[0]*x[1]]])

    return J

def Newton(x0,tol,Nmax):

    ''' inputs: x0 = initial guess, tol = tolerance, Nmax = max its'''
    ''' Outputs: xstar= approx root, ier = error message, its = num its'''

    for its in range(Nmax):
       J = evalJ(x0)
       Jinv = inv(J)
       F = evalF(x0)

       x1 = x0 - Jinv.dot(F)

       if (norm(x1-x0) < tol):
           xstar = x1
           ier =0
           return[xstar, ier, its]

       x0 = x1

    xstar = x1
    ier = 1
    return[xstar,ier,its]
```

In [21]:
```
root, err, iters = Newton([1,1], tol, max_iter)

print(f"root: {root}, error: {err}, iterations: {iters}")
```

```
root: [0.5       0.8660254], error: 0, iterations: 3
```

This converges much more quickly in terms of iteration count.

> (d)

$(x, y) = (.5, 0.8660254)$

$f(x, y) = 3x^2 - y^2 = 3(.5)^2 - (0.8660254)^2 = 3(.25) - 0.7499999934 = .75 - 0.7499999934 = 6.6 \times 10^{-9}$

$g(x, y) = 3xy^2 - x^3 - 1 = 0 = 3(.5)(0.8660254)^2 - (.5)^3 - 1 = -9.83226 \times 10^{-9}$

These results are very near 0 and we can increase tolerance if needed.

> ## 2.
>
> Given $\begin{cases} g_1 = \frac{1}{\sqrt{2}}\sqrt{1 + (x+y)^2} - \frac{2}{3} \\ g_2 = \frac{1}{\sqrt{2}}\sqrt{1 + (x-y)^2} - \frac{2}{3} \end{cases}$

$|\frac{\partial g_i(x)}{\partial x_j}| \leq \frac{K}{n}$

$x^{(k)} = G(x^{(k-1)})$, for each $k \geq 1$ converges to the unique fixed point $p \in D$ and $||x^{(k)} - p||_\infty \leq \frac{K^k}{1-K}||x^{(1)} - x^{(0)}||_\infty$

Find a region D in the x, y–plane for which the fixed point iteration $\begin{cases} x_{n+1} = \frac{1}{\sqrt{2}}\sqrt{1 + (x_n + y_n)^2} - \frac{2}{3} \\ y_{n+1} = \frac{1}{\sqrt{2}}\sqrt{1 + (x_n - y_n)^2} - \frac{2}{3} \end{cases}$ is guaranteed to converge to a unique solution for any starting point $(x_0, y_0) \in D$.

We first show the sum of the partial derivatives is less than some constant K < 1.

$\frac{\partial g_1}{\partial x} = \frac{x+y}{\sqrt{2}\sqrt{1+(x+y)^2}}$, $\frac{\partial g_1}{\partial y} = \frac{x+y}{\sqrt{2}\sqrt{1+(x+y)^2}}$, gives $\frac{\partial g_1}{\partial x} + \frac{\partial g_1}{\partial y} = \sqrt{2}\frac{x+y}{\sqrt{1+(x+y)^2}}$

$\frac{\partial g_2}{\partial x} = \frac{x-y}{\sqrt{2}\sqrt{1+(x-y)^2}}$, $\frac{\partial g_2}{\partial y} = \frac{x-y}{\sqrt{2}\sqrt{1+(x-y)^2}}$, gives $\frac{\partial g_2}{\partial x} + \frac{\partial g_2}{\partial y} = \sqrt{2}\frac{x-y}{\sqrt{1+(x-y)^2}}$

letting u = (x+y) and v = (x-y), $K/2 = \sqrt{2}\frac{u}{\sqrt{1+(u)^2}} < \frac{1}{2}$ and $K/2 = \sqrt{2}\frac{v}{\sqrt{1+(v)^2}} < \frac{1}{2}$ gives $|x+y| < 1$ and $|x-y| < 1$

For $D = [-R, R] \times [-R, R]$, the max of $x, y \in [-R, R]$ is $x + y = 2R$ and $x - y = 2R$

If we choose a value for R such that plugging the max value (2R) into $K = \sqrt{2}\frac{u}{\sqrt{1+(u)^2}} = \sqrt{2}\frac{2R}{\sqrt{1+(2R)^2}} < 1$

$2\frac{(2R)^2}{1+(2R)^2} < 1$ gives $\frac{(2R)^2}{1+(2R)^2} < \frac{1}{2}$ gives $\frac{4R^2}{1+4R^2} < \frac{1}{2}$

Letting R = .1 gives $K = \sqrt{2}\frac{2(.1)}{\sqrt{1+(2(.1))^2}} = \sqrt{2}\frac{.2}{\sqrt{1+.2^2}} = \sqrt{2}\frac{.2}{\sqrt{1+.04}} = \sqrt{2}\frac{.2}{\sqrt{1.04}} = 0.2773500981 < 1$

Then $D = [-.1, .1] \times [-.1, .1]$ will guarantee convergence.

3.

$$\begin{cases} x_{n+1} = x_n - df_x \\ y_{n+1} = y_n - df_y \end{cases}$$

$d = \frac{f(x_n, y_n)}{(f_x^2 + f_y^2)}$

> (a)

To move from (x_n, y_n) off curve to a point on the curve along the normal line, the displacement vector must be parallel to the gradient vector

$\Delta f = [f_x \ f_y]^T$ gives $x_{n+1} - x_n = \lambda f_x$ and $y_{n+1} - y_n = \lambda f_y$

Using a first order Taylor expansion around (x_n, y_n) for $f(x_{n+1}, y_{n+1}) = 0$ gives

$f(x_{n+1}, y_{n+1}) = f(x_n, y_n) + f_x(x_{n+1} - x_n) + f_y(y_{n+1} - y_n) = 0$

$f(x_n, y_n) + f_x(\lambda f_x) + f_y(\lambda f_y) = f(x_n, y_n) + \lambda(f_x)^2 + \lambda(f_y)^2 = f(x_n, y_n) + \lambda(f_x^2 + f_y^2) = 0$

$\lambda = -\frac{f(x_n, y_n)}{(f_x^2 + f_y^2)} = -d$

gives

$x_{n+1} = x_n + \lambda f_x = x_n - df_x$

$y_{n+1} = y_n + \lambda f_y = y_n - df_y$

> (b)

In [25]:
```
x, y, z = 1, 1, 1
max_iters = 20
tol = 10e-10

for i in range(max_iters):
    f_val = x**2 + 4 * y**2 + 4 * z**2 - 16

    fx = 2*x
    fy = 8*y
    fz = 8*z

    print(f"For step {i}: ({x}, {y}, {z}) gives {f_val}")

    if abs(f_val) < tol:
        print("Convergence")
        break

    d = f_val / (fx**2 + fy**2 + fz**2)

    x = x - d * fx
    y = y - d * fy
    z = z - d * fz
```

```
For step 0: (1, 1, 1) gives -7
For step 1: (1.106060606060606, 1.4242424242424243, 1.4242424242424243) gives 1.4511019283746585
For step 2: (1.0939261558696716, 1.3617416944918586, 1.3617416944918586) gives 0.03139797463626337
For step 3: (1.0936424626088905, 1.3603291057551175, 1.3603291057551175) gives 1.6043737378623746e-05
For step 4: (1.093642317388233, 1.3603283832232336, 1.3603283832232336) gives 4.199307568342192e-12
Convergence
```

This converges quadratically as the magnitude of the exponent of the residuals continues to roughly double each step.